

DÉPLOIEMENT D'UN SYSTÈME DE MESSAGERIE SÉCURISÉE



Aristide Wafo - Antoine Douay
01/06/2026

Table des matières

1. Description du Projet et Fondations Cryptographiques	2
1.1 Contexte et Objectifs du Projet	2
1.2 Concepts Cryptographiques Fondamentaux	2
2. Architecture Technique et Flux de Travail	3
2.1 Flux de Provisioning et de Gestion des Identités	3
2.2 Flux de Chiffrement et d'Émission des Messages	3
2.3 Flux de Déchiffrement et de Réception des Messages	4
3. Guide de l'Utilisateur et Documentation Technique	5
3.1 Initialisation Manuelle de l'Autorité de Certification	5
3.2 Automatisation du Provisioning : <code>creer_utilisateur.sh</code>	5
3.3 Chiffrement et Signature à l'Émission :	6
3.4 Déchiffrement et Authentification à la Réception :	7
4. Bilan Synthétique et Perspectives d'Évolution	7
4.1 Enseignements Clés du Projet	7
4.2 Recommandations et Perspectives d'Amélioration	8

1. Description du Projet et Fondations Cryptographiques

1.1 Contexte et Objectifs du Projet

Dans un environnement numérique où la compromission des données et le détournement des communications sont devenus des menaces systémiques, la sécurisation des flux constitue un enjeu critique pour les organisations. Ce projet s'inscrit dans une démarche de conception et d'implémentation d'une infrastructure locale de messagerie hautement sécurisée, développée intégralement à l'aide d'outils open source éprouvés : GnuPG (GPG) et OpenSSL. L'objectif principal est d'appliquer les principes fondamentaux de la cryptographie pour bâtir un écosystème étanche capable de garantir la confidentialité, l'authenticité et l'intégrité des données échangées entre collaborateurs.

Au-delà de la simple manipulation manuelle de commandes cryptographiques, ce travail vise à automatiser la gestion du cycle de vie des utilisateurs. Cela inclut l'orchestration technique de la création de répertoires isolés, la génération automatique de paires de clés asymétriques, et la mise en place d'une Infrastructure à Clés Publiques (PKI) complète. Le système final permet aux utilisateurs d'écrire et de lire des messages dont la sécurité n'est pas dépendante du canal de transmission, mais repose entièrement sur la robustesse des algorithmes de chiffrement et de signature mis en œuvre.

1.2 Concepts Cryptographiques Fondamentaux

Pour atteindre le niveau de sécurité exigé, l'architecture logicielle s'appuie sur une combinaison de trois piliers de la cryptographie moderne.

La Cryptographie Asymétrique :

Contrairement au chiffrement symétrique qui utilise une clé unique partagée pour verrouiller et déverrouiller une information, la cryptographie asymétrique repose sur un couple de clés mathématiquement liées : une clé publique et une clé privée. La clé publique est conçue pour être distribuée librement à tous les correspondants potentiels sans compromettre la sécurité. Elle permet à n'importe quel émetteur de chiffrer un message destiné au propriétaire de la paire de clés. La clé privée, quant à elle, doit impérativement rester sous le contrôle exclusif de son propriétaire. Elle seule détient la capacité algorithmique de déchiffrer les données qui ont été verrouillées par la clé publique correspondante. Ce mécanisme résout le problème historique de la distribution des clés de chiffrement sur un canal non sécurisé.

Les Signatures Numériques et le Hachage :

La confidentialité ne suffit pas à sécuriser un échange, il est tout aussi crucial de s'assurer que l'identité de l'émetteur est légitime (authenticité) et que le message n'a subi aucune altération durant son transit (intégrité). Pour ce faire, le système applique d'abord une fonction de hachage cryptographique (via la bibliothèque OpenSSL) sur le message brut pour en extraire une empreinte numérique de taille fixe. Cette empreinte possède une propriété d'unicité : la moindre modification d'un seul caractère dans le message original modifierait radicalement le hash résultant.

Cette empreinte est ensuite chiffrée à l'aide de la clé privée de l'expéditeur, générant ainsi la signature numérique (via GnuPG). Le destinataire utilise la clé publique de l'émetteur pour déchiffrer cette signature et recalculer le hash du message reçu. Si les deux empreintes concordent, le destinataire a la certitude absolue que le message provient du bon auteur et qu'il n'a pas été modifié.

L'Infrastructure à Clés Publiques (PKI) et les Certificats

Le déploiement de clés publiques à grande échelle introduit le risque de l'attaque de man in the middle, où un attaquant pourrait substituer sa propre clé publique à celle du destinataire légitime. Pour contrer cette faille, nous mettons en place une Autorité de Certification (CA) interne à l'aide d'OpenSSL. La CA fait office de tiers de confiance. Elle valide l'identité des utilisateurs du réseau en signant numériquement leur certificat électronique. Le certificat associe de manière indiscutable l'identité d'un utilisateur (son nom, son adresse e-mail) à sa clé publique, le tout scellé par la signature de la CA.

2. Architecture Technique et Flux de Travail

L'architecture du système de messagerie sécurisée est modulaire et s'articule autour d'interactions précises entre le système de fichiers Linux, l'utilitaire GnuPG et les modules de gestion de certificats OpenSSL. Le fonctionnement global se décompose en trois flux opérationnels distincts.

2.1 Flux de Provisioning et de Gestion des Identités

L'initialisation de l'infrastructure commence par la création de la CA racine, caractérisée par sa clé secrète durcie `ca.key` et son certificat racine auto-signé `ca.crt`. Une fois cette entité établie, la création de chaque nouvel utilisateur est entièrement automatisée par un script.

Lorsqu'un profil est instancié, le système génère une arborescence de répertoires isolée dans le stockage local de la machine. À l'intérieur de ce périmètre de sécurité, deux paires de clés distinctes sont créées : un jeu de clés GPG configuré pour l'application des mécanismes de chiffrement et de signature des correspondances, et une clé privée OpenSSL. Le script extrait la clé publique GPG de l'utilisateur sous la forme d'un bloc de texte armuré ASCII afin de la rendre exportable et partageable avec l'ensemble des futurs correspondants du réseau. En parallèle, une demande de signature de certificat (CSR) est assemblée avec les attributs de l'utilisateur puis soumise à la CA racine. La CA traite cette demande, applique sa propre signature cryptographique et génère le certificat X.509 final, garantissant l'identité de l'utilisateur au sein du système.

2.2 Flux de Chiffrement et d'Émission des Messages

Le processus d'écriture et de sécurisation d'une correspondance (initié par le script `ecrire_message.sh`) combine de manière transparente le hachage, la signature et le chiffrement asymétrique.

Lorsqu'un expéditeur rédige un message à destination d'un tiers, le flux technique se déroule selon la séquence suivante :

Le texte brut est temporairement consigné dans un espace tampon isolé du système de fichiers afin d'être traité par le moteur cryptographique.

Le système configure un environnement GnuPG éphémère et étanche en exploitant des variables d'environnement restrictives. Cet environnement importe à la volée la clé privée de l'expéditeur et la clé publique du destinataire ciblé.

GnuPG effectue une double opération en une seule passe : il calcule l'empreinte du fichier tampon, injecte la signature numérique de l'expéditeur à l'aide de sa clé privée, puis chiffre l'intégralité du package (le texte original accompagné de sa signature) en utilisant la clé publique du destinataire.

Le résultat est exporté sous la forme d'un fichier ASCII portant l'extension .gpg. Ce document sécurisé est alors transféré directement dans l'espace de stockage ou la "boîte aux lettres" du destinataire, tandis que le fichier tampon contenant le texte en clair est immédiatement écrasé et purgé du disque pour des raisons de sécurité.

2.3 Flux de Déchiffrement et de Réception des Messages

Lorsqu'un utilisateur souhaite prendre connaissance des messages qui lui ont été adressés, le script de lecture (lire_message.sh) exécute le cheminement algorithmique inverse pour restituer l'information tout en mesurant sa conformité sécuritaire.

Le traitement de lecture respecte les étapes suivantes :

Le script cible le fichier chiffré .gpg présent dans le répertoire de l'utilisateur connecté.

Un environnement GnuPG isolé est de nouveau instancié pour faire barrière aux interférences du système global. Cet environnement procède à l'importation de la clé privée du destinataire (indispensable pour briser le chiffrement) et de la clé publique de l'expéditeur présumé (indispensable pour l'analyse de l'identité).

GnuPG initie l'opération de déchiffrement à l'aide de la clé privée de l'utilisateur récepteur. Si la clé correspond, l'enveloppe cryptographique est rompue, révélant le texte d'origine ainsi que la signature de l'émetteur.

Immédiatement après, le moteur extrait la signature numérique et l'analyse en utilisant la clé publique de l'expéditeur. GnuPG compare dynamiquement les hashes et émet son verdict sur la sortie standard du terminal. Le système affiche alors de manière distincte le statut de validité de la signature (confirmant que l'intégrité et l'authenticité sont préservées) ainsi que le corps du message en clair pour l'utilisateur final.

3. Guide de l'Utilisateur et Documentation Technique

L'implémentation opérationnelle du système de messagerie sécurisée nécessite une compréhension des commandes système sous-jacentes. Cette section documente les procédures d'exploitation, le rôle des scripts Bash développés et l'analyse de leur logique interne.

3.1 Initialisation Manuelle de l'Autorité de Certification

L'établissement de la confiance au sein de l'architecture logicielle requiert la création préalable et manuelle de l'entité racine. Cette phase s'exécute directement dans le terminal de la machine de test Kali Linux et pose les fondations de la PKI.

En premier lieu, l'administrateur doit structurer l'espace de travail en isolant les répertoires de la CA de ceux destinés aux futurs utilisateurs du service. Une fois cette arborescence logicielle créée, la génération de la clé privée de la CA s'effectue au moyen de l'algorithme RSA configuré avec une longueur de clé de 4096 bits. Cette clé est assortie d'un chiffrement symétrique AES-256 afin d'interdire toute lecture directe en clair sur le disque sans la saisie préalable d'une phrase de passe complexe.

À partir de cette clé confidentielle, l'administrateur génère le certificat public de l'autorité racine (le fichier ca.crt). Ce certificat est dit auto-signé car l'émetteur et le sujet désigné sont identiques. Il est configuré pour une période de validité de 365 jours en utilisant l'algorithme de hachage SHA-256 pour sceller l'intégrité de ses attributs institutionnels (tels que le nom de l'organisation ou la localisation géographique).

```
(kali@kali)-[~/Projet_Messagerie/ca]
$ openssl genrsa -aes256 -out ca.key 4096
Enter PEM pass phrase:
Verifying - Enter PEM pass phrase:

(kali@kali)-[~/Projet_Messagerie/ca]
$ openssl req -x509 -new -nodes -key ca.key -sha256 -days 365 -out ca.crt
Enter pass phrase for ca.key:
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:FR
State or Province Name (full name) [Some-State]:IDF
Locality Name (eg, city) []:PARIS
Organization Name (eg, company) [Internet Widgits Pty Ltd]:SUPINFO
Organizational Unit Name (eg, section) []:SUPINFO
Common Name (e.g. server FQDN or YOUR name) []:.
Email Address []:supinfo@local.com
```

3.2 Automatisation du Provisioning : creer_utilisateur.sh

La gestion des identités à l'échelle de l'organisation ne pouvant s'effectuer manuellement, le script creer_utilisateur.sh résout cette problématique en prenant en paramètre unique le nom de l'identité à créer.

```

(kali@kali)-[~/Projet_Messagerie]
$ ./creer_utilisateur.sh bob
=== Création de l'utilisateur : bob ===
Génération des clés GPG ...
gpg: A key for "bob@ecole.local" already exists
Génération de la clé privée OpenSSL ...
Création de la demande de certificat (CSR) ...
Signature du certificat par la CA ...
Certificate request self-signature ok
subject=C=FR, ST=IdF, L=Paris, O=Ecole, CN=bob
Enter pass phrase for ./ca/ca.key:
=== Utilisateur bob créé avec succès ! ===

```

Sur le plan technique, le script initialise d'abord le répertoire de l'utilisateur. Il invoque ensuite le moteur GnuPG en mode non interactif (--batch) afin de générer une paire de clés asymétriques RSA de 2048 bits associés à l'adresse de messagerie institutionnelle de l'utilisateur. Pour garantir la portabilité des données, le script extrait immédiatement la clé publique et la clé privée sous forme de blocs de texte ASCII.

Dans un second temps, le script bascule sur l'écosystème OpenSSL pour générer la clé privée de l'utilisateur dédiée aux authentifications de certificats. Il formalise une demande de signature de certificat (CSR) contenant les métadonnées de l'agent. Enfin, le script sollicite la clé secrète de la CA pour signer cette demande. Le certificat X.509 résultant (le fichier .crt) est déposé dans le répertoire utilisateur, achevant ainsi le processus de provisioning réglementaire.

3.3 Chiffrement et Signature à l'Émission :

L'envoi d'une correspondance sécurisée implique la mise en œuvre conjointe des mécanismes de confidentialité et de non-répudiation. Le script `ecrire_message.sh` prend trois arguments : l'expéditeur, le destinataire et le texte du message.

```

(kali@kali)-[~/Projet_Messagerie]
$ ./ecrire_message.sh bob alice "salut alice, comment tu vas ?"
=== Composition d'un message sécurisé de bob pour alice ===
Chiffrement et signature en cours (Environnement Isolé) ...
Message signé, chiffré et déposé dans le dossier de alice !

```

La contrainte technique majeure de ce script réside dans l'isolation du trousseau de clés GPG sous Kali Linux. Pour empêcher toute interférence avec le système global, le script crée un répertoire temporaire et redéfinit la variable d'environnement GNUPGHOME. Il importe au sein de cet espace étanche la clé privée de l'émetteur et la clé publique du récepteur. Le texte brut est écrit dans un fichier temporaire.

Le script appelle ensuite la commande de chiffrement et de signature combinée de GnuPG. L'option --sign ordonne au moteur de calculer le hash du message et de le sceller avec la clé privée de l'expéditeur. L'option --encrypt chiffre le fichier résultant avec la clé publique du

destinataire. Le document final (.gpg) est déposé directement dans l'espace du destinataire. Par mesure de sécurité, le fichier en clair et le trousseau temporaire sont immédiatement détruits à l'aide de commandes de purge du système de fichiers.

3.4 Déchiffrement et Authentification à la Réception :

La lecture d'un message nécessite l'exécution inverse des opérations cryptographiques afin de restituer l'information tout en statuant sur sa légitimité. Le script `lire_message.sh` requiert le nom du lecteur et celui de l'auteur présumé.

```
(kali㉿kali)-[~/Projet_Messagerie]
$ ./lire_message.sh alice bob
=== Lecture et Vérification du message pour alice ===

— Résultat du Déchiffrement et de la Signature —
gpg: encrypted with cv25519 key, ID 5C310EE0A3777A50, created 2026-05-28
      "alice@ecole.local"
salut alice, comment tu vas ?
gpg: Signature made Tue 02 Jun 2026 08:53:33 AM EDT
gpg:          using EDDSA key 4D1D34542210C0B586013186802E0A30B16C24AF
gpg:          issuer "bob@ecole.local"
gpg: Good signature from "bob@ecole.local" [unknown]
gpg: WARNING: Using untrusted key!
```

À l'instar du script d'émission, la sécurité de l'opération repose sur l'instanciation d'un environnement GNUPGHOME éphémère. Le script y charge la clé privée du destinataire (nécessaire au déchiffrement) et la clé publique de l'expéditeur (nécessaire à la vérification).

Lors de l'appel à la fonction `--decrypt`, GnuPG utilise la clé privée du récepteur pour casser l'enveloppe de confidentialité. Une fois le message ouvert, le moteur extrait la signature numérique embarquée et applique la clé publique de l'expéditeur pour valider l'intégrité de l'empreinte. La sortie standard du terminal affiche simultanément le corps du message en clair et le rapport de conformité de la signature. Si la signature est déclarée valide, l'utilisateur a la garantie technique que le message n'a subi aucune altération et provient bien du collaborateur authentifié.

4. Bilan Synthétique et Perspectives d'Évolution

4.1 Enseignements Clés du Projet

La réalisation de cette infrastructure de messagerie sécurisée a permis de confronter les théories mathématiques de la cryptographie aux réalités techniques d'un système d'exploitation Linux. L'intégration conjointe de GnuPG et d'OpenSSL démontre que la sécurité informatique ne repose pas sur un outil unique, mais sur l'empilement cohérent de plusieurs briques technologiques complémentaires.

Ce projet met en évidence qu'une gestion rigoureuse des clés publiques et privées est indispensable : l'erreur humaine ou une mauvaise configuration des droits d'accès sur le système de fichiers suffit à compromettre l'intégralité d'un canal sécurisé. L'automatisation par scripts Bash illustre également les contraintes du développement sécurisé, notamment l'obligation de nettoyer les fichiers temporaires en clair et d'isoler les processus d'exécution pour interdire les fuites de données en mémoire.

4.2 Recommandations et Perspectives d'Amélioration

Bien que l'architecture actuelle réponde strictement aux exigences du cahier des charges en simulant les échanges au sein du système de fichiers local, plusieurs axes d'évolution permettraient de transformer ce prototype académique en une solution industrielle viable :

Acheminement Réseau via un Serveur de Transport (Postfix) : L'amélioration prioritaire consiste à interconnecter nos scripts avec un agent de transfert de courrier tel que Postfix. Au lieu de déposer les fichiers .pgp dans des répertoires Unix locaux, les scripts exploiteraient le protocole SMTP pour acheminer les messages chiffrés à travers un réseau local, simulant ainsi un véritable trafic d'entreprise.

Intégration d'une Interface Graphique Utilisateur (Thunderbird) : Pour rendre la technologie accessible à des profils non techniques, l'importation des paires de clés asymétriques et des certificats X.509 au sein du client de messagerie Mozilla Thunderbird constituerait une évolution majeure. Le module OpenPGP natif de Thunderbird prendrait alors en charge les phases de chiffrement et de déchiffrement de manière totalement transparente pour l'opérateur.

Durcissement de la Gestion des Clés (Gestion des Passphrases) : Afin de faciliter l'automatisation, les clés GPG des utilisateurs ont été générées sans phrase de passe protectrice. Une implémentation de production exigerait la mise en place de clés privées chiffrées, combinée à l'utilisation d'un démon de sécurité tel que gpg-agent, capable de mémoriser temporairement les clés en mémoire vive de façon sécurisée sans exposer les secrets sur le support de stockage.